

5/1/26

Parallel Computing and Architecture (PCAP)

Modules

top500.org $\Rightarrow$ super-comp.

- ch1. Concepts of PCAP
- ch2. MPI programs
- ch3. OpenCL & Cuda
- ch4. Cuda Optimization
- ch5. Parallel Image Processing

Assignments :- 20 mks (Quiz :- In class Quiz) } 50 mks Internals
Midterm :- 30 mks

Mini Project :- for IET Accreditation

Lab :- 10 Labs (each 3 mks, 2 mks behavior)

%p :- MPI, Cuda, General Programs

Lab 5 :- Observation 1, MPI + Cuda programs, kernel (Viva)

Quiz :- 10 mks (end of Lab)

Internals = 60 mks, Externals = 40 mks (Endsem)

Documentation :- Code (MPI/Cuda) Screenshot w. Observ.

Lab 6 :- Midterm Examination (20 mks)

Module 1: Introduction to MPI

→ Programming $\Rightarrow$ ① Implicit
② Explicit

→ Basic terms $\Rightarrow$ ① Parallel programming models

→ Message passing

→ Shared Memory (not there)

② Message passing programming

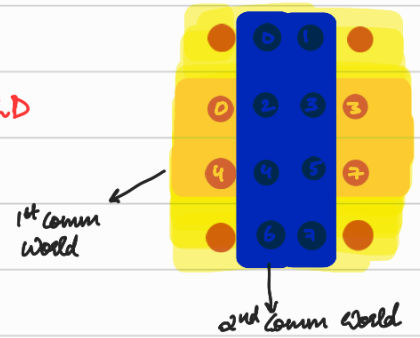
→ Message passing libraries $\Rightarrow$ ① PVM (Parallel Virtual Machine)

② MPI (Message Passing Interface)

- creates process (to divide jobs)
- run program in clusters
- has no threads

→ **MPI_COMM_WORLD**: default process grp consisting of such (elastic) process. (default communication world)

- `MPI_Init(&argc, &argv)`: Initializing MPI
(-np, value)
- `MPI_Comm_size(comm, &size)`: Process size → **MPI_COMM_WORLD**
- `MPI_Comm_rank(comm, &rank)`: Process ranking
- `MPI_Finalize()`: last func. to call to close the environment



→ Run :- `filename.c -o filename.out`

→ 7/1/25
Compile :- `-np 4 filename.out` → this can be changed based on no. of processes
just a flag

→ MPI is not a programming language
→ pass info to communicate & synchronize messages

→ Advantages :- ① run well widely on MMD architectures

→ Datatype for MPI representation :-

- ① MPI_INT
- ② MPI_LONG
- ③ MPI_FLOAT
- ④ MPI_SHORT
- ⑤ MPI_CHAR

→ int ierr :- tells line & error @ that line decoded in MPI library.

12/1/26

→ Types of Communication :-

① Point to Point

→ `MPI_Send`

`MPI_Send (void *message, //addr. of data to be transmitted
int count, //no. of data lines`

for MPI_Recv `MPI_Datatype datatype, //type of data to be trans.
int source, → int dest, //rank of the process to recv the data
int tag, //integer label for the message
MPI_Comm com) //communicator`

`MPI_Status *status →`

//status info. of data recvd, default is **MPI_STATUS_IGNORE**

- internal buffering
- no synchronization forced
- just goes to Recv
- debugging need: lower

→ MPI_send → Synchronous Send

- no internal buffering
- blocks until Recv
- pure synchronization
- Debugging need: Higher

MPI_Send (void *message, //addr. of data to be transmitted

int count, //no. of data lines

MPI_Datatype datatype, //type of data to be trans.

int dest, //rank of the process to recv the data

int tag, //integer label for the message

MPI_Comm com) //communicator

→ MPI_Bsend → Buffered Send

MPI_Bsend (void *message, //addr. of data to be transmitted

int count, //no. of data lines

MPI_Datatype datatype, //type of data to be trans.

int dest, //rank of the process to recv the data

int tag, //integer label for the message

MPI_Comm com) //communicator

→ Must be used with the MPI_Buffer_attach() and MPI_Buffer_detach() routines.

MPI_Buffer_attach (void *buffer,
int size)

MPI_Buffer_detach (void *buffer,
int size)

→ MPI_BSEND_OVERHEAD : to get the header, its from the Environment
hence is written in capital.

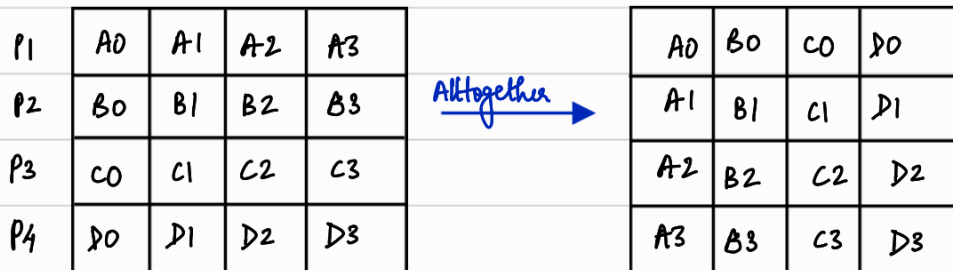
→ free(buffer) : When using malloc we need to free the dynamically allocated
memory. //will not be visible to write in the memory if not
freed.

→ Deadlock

14/1/26



Collective Communication in MPI



* MPI_Barrier (MPI_COMM_WORLD);

19/1/26



Collective MPI's



MPI_Bcast

`MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);`

- used to send same data from one process (root) to all other processes in a communicator.
- only the root process provides the actual data; others receive it.
- Blocking call: it returns only after broadcast is complete
- commonly used for i/p values, configuration data & constants



MPI_Scatter

- Used to divide and distribute chunks of data from one root process to all processes in a communication.
- The root process holds the full data, split it to equal-sized parts.
- Each process (including root) receives only its own portion.
- Blocking call: completes only after data is distributed.

fact: question → full works: reduce, partial works: allgather



Types of Collective Operations

①

Synchronization

processes wait until all members of the group have reached the synchronization point.



MPI_Scatter

→ Row Scatter

→ Used to distribute rows of a 2D array (matrix) from the root process to all processes.

→ Each process receives one or more complete rows.

→ Data is stored row-wise, so rows are contiguous in memory.

→ Typically implemented with MPI_Scatter by sending columns elements by rows.

→ Common in, matrix operations (like 11th matrix multiplication)

→ Column Scatter

→ Used to distribute

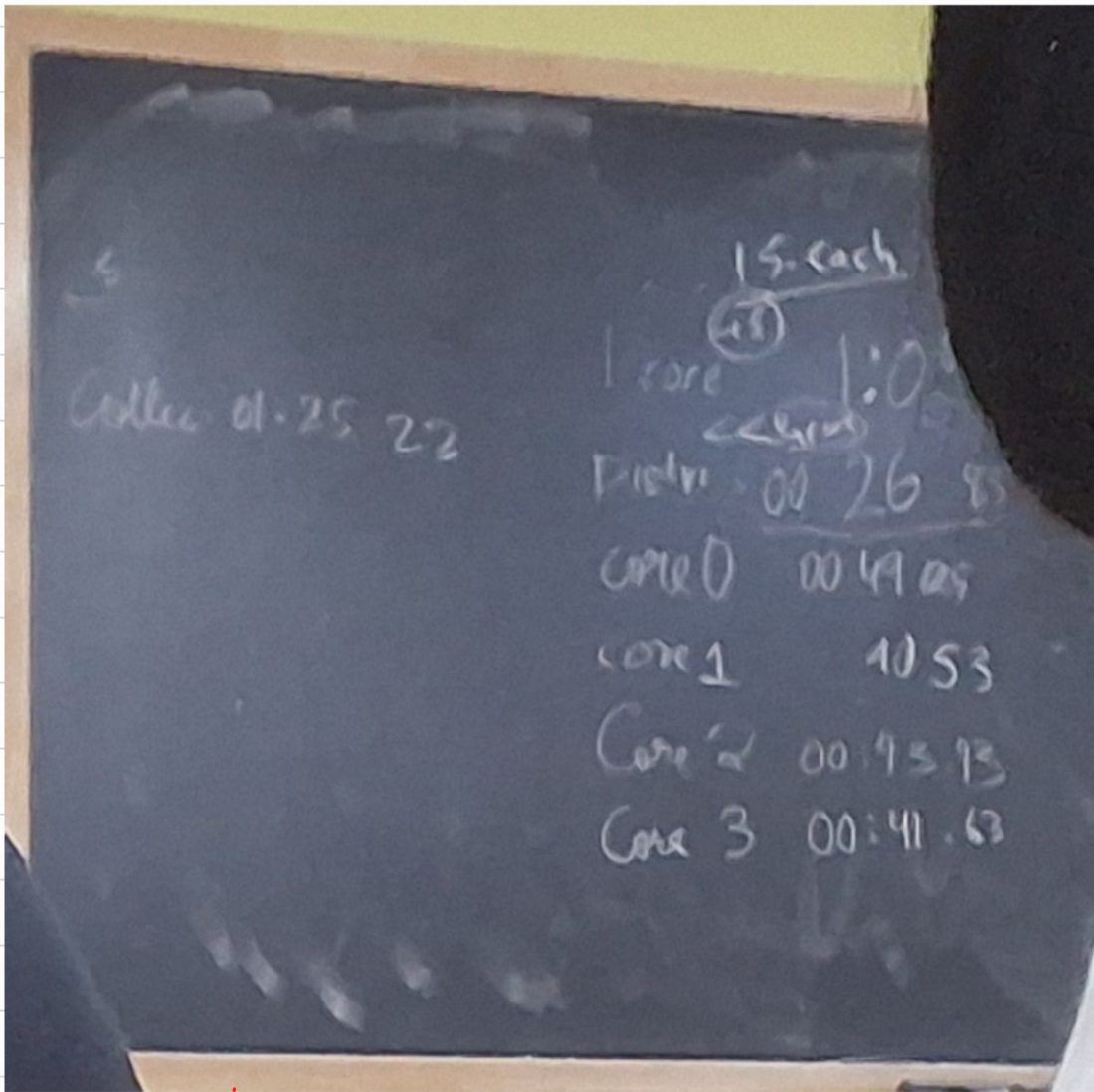
MPI-Scan
MPI-Error-handling
functions

Module 2: Introduction to Parallel Architectures

temporal → behavioral
spatial → together: data

28/1/20

Candy Sorting (Parallel vs single)



31/1/26



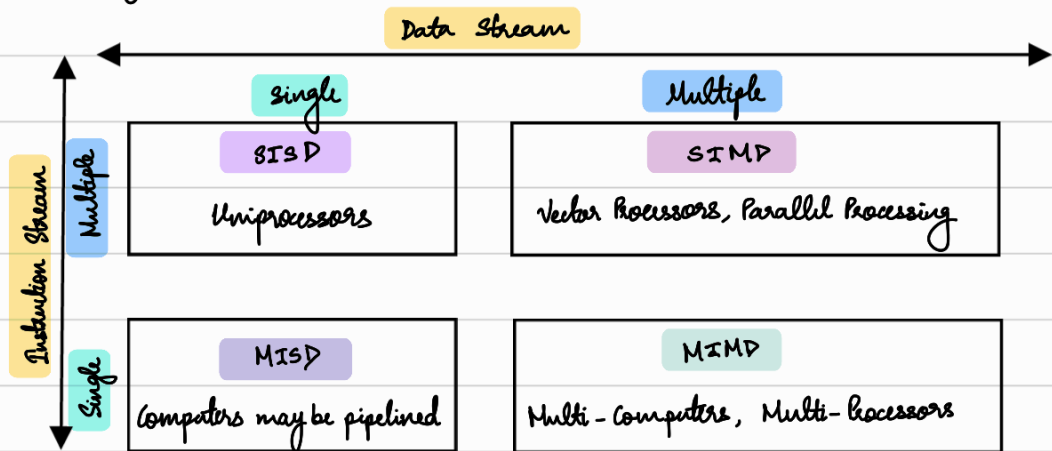
Hardware architecture of parallel computing :-

- ① Single-instruction, single-data (SISD) systems
- ② Single-instruction, Multiple-data (SIMD) systems
- ③ Multiple-instruction, single-data (MISD) systems
- ④ Multiple-instruction, Multiple-data (MIMD) systems

Over large scale parallel programming is possible.



Flynn's Taxonomy



GPU as Parallel Computers :-

article : Fare lunch is over [Intel bankruptcy story, hedgenization]

GPU - Manycore vs CPU - multicore ??

2/2/26

Module 3: Introduction to CUDA Programming

- gpgpu
- NVIDIA releasing CUDA (Compute Unified Device Architecture)
- warp & threads
-

Module 4: CUDA Programming



CUDA program structure

- kernel launch: large no. of threads
- h_C: host computer
- cuda runtime provides API



$i = \text{blockIdx} \cdot x * \text{blockDim} \cdot x + \text{threadIdx} \cdot x \Rightarrow$ Vary by 2D, 3D arrays...

CUDA Thread Organization structure

$$\text{grid} = (3, 1, 1) = 3 \text{ blks} \quad \left. \begin{array}{l} \\ \end{array} \right\} 8 \times 3 = \underline{24 \text{ blks}}$$

$$\text{blocks} = (4, 2, 2) = 8 \text{ blks}$$

structure given

by this layout 4x2 $\rightarrow$ [w. 8 threads in each blk]

$$128 \times 32 = \underline{4096} \rightarrow$$

* dimension of grid $\rightarrow$ gridDim

* dimension of each blk $\rightarrow$ blockDim

All blocks in a grid have the same dimension

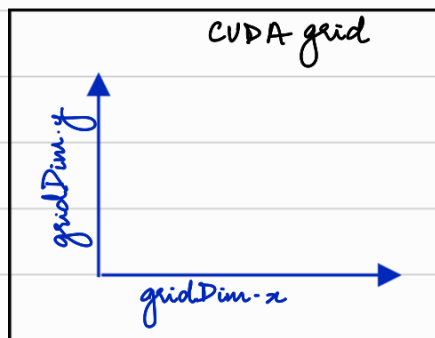
$\rightarrow$ range from 1 - 65536

total size of a block is limited to 1024 threads as long as the total no. of thread does not exceed 1024.

Eg:- $(512, 2, 1) \rightarrow$ Valid $(512 \times 2 \times 1 = 1024)$

$(1024, 1, 1) \rightarrow$ Valid $(1024 \times 1 \times 1 = 1024)$

$(82, 32, 2) \rightarrow$ Invalid exceeds by 2048 $(82 \times 32 \times 2)$



Q $\langle\langle 4, 3 \rangle\rangle$

A: block $\rightarrow (4, 1, 1)$

grid $\rightarrow$

tot. no of threads $= 4 \times 3 = 12$ threads

blockDim.x $= 3$

gridDim.x $= 4$

Q) $globalid = t_{127}$ then $t_{x,y,z} = ?$
 $dim\ grid(4, 2, 1)$
 $dim\ block(4, 4, 2)$

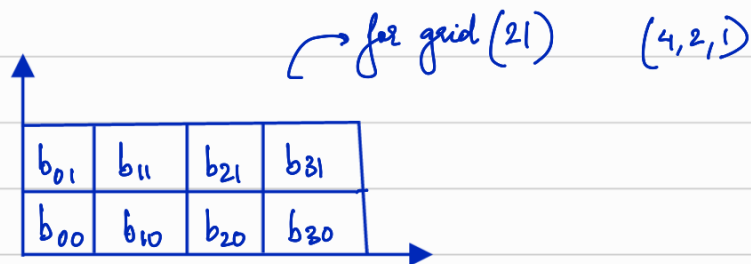
6

A: $globalid = t_{127}$ 2, 4

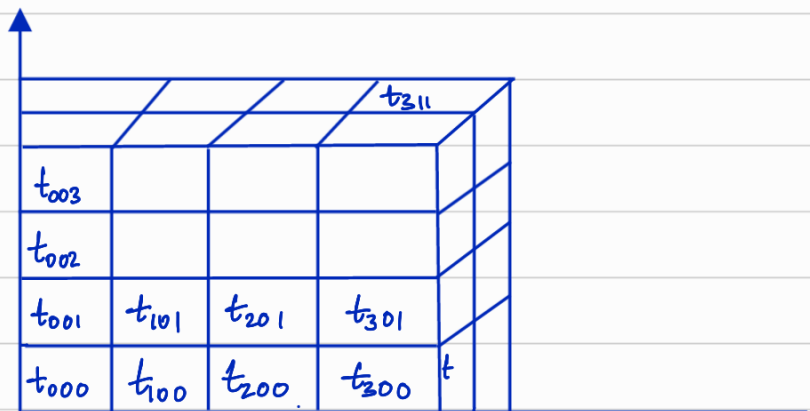
$$threads/blk = 4 \times 4 \times 2 = 32$$

$$blkid = 127 / 32 = 3$$

$$local\ thread\ id = 127 \% 32 = 31$$



Now, for (4, 4, 2)



32

grid (4, 3, 3)
 blk (8, 4, 9)
 1296th thread & blk

$$4 \times 7 + 3$$

$$\begin{array}{r} 28 \\ 3 \Rightarrow 72 \end{array}$$

3

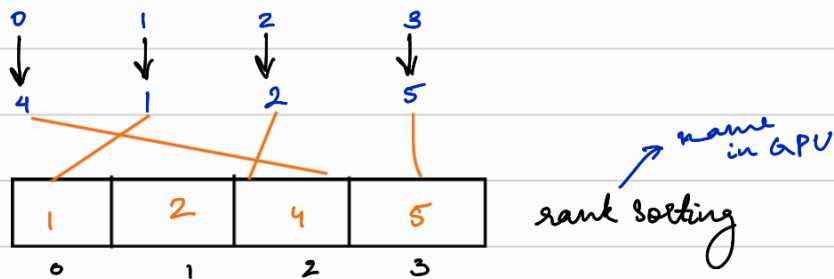
1

11/2/26
→

calculations of global thread()

14/2/26

Solution sort A = [29, 10, 14, 37, 13]



Odd-Even Transposition sort

i/p = 7 8 6 5 4 3 2 1

iteration 1: Even Phase

thread	compared index	values	swap	
0	(0,1)	(7,8)	no	7 8 6 5 4 3 2 1
1	(2,3)	(6,5)	yes	7 8 5 6 4 3 2 1
2				
3				

→ `CudaDeviceSynchronize()`:

16/2/26

Mapping threads to Multidimensional Data

Matrix multiplication:-

- ① Row major
- ② Column major

} architecture vary for memory allocation of $A_{00}, A_{01}, A_{10}, \dots$

→ cache miss / cache hit

→ flattening a 2D-array

Q) $\text{Grid} \langle \langle \langle 5, 4, 1 \rangle \rangle \rangle$, $\text{global threadid} = 190$
 $\text{Blk} \langle \langle \langle 8, 4, 2 \rangle \rangle \rangle$

index of 190th thread, t_x, t_y & t_z values = ?

A: threads per blk = $8 \times 4 \times 2 = 64$
 $\text{blkid} = \frac{190}{64} = 2.969 \approx \underline{\underline{2}}$

$\text{grid.x} = 5$

$\text{blkidx.x} = 2 \bmod 5 = 2$

$\text{blkidx.y} = 2/5 = 0.4 \approx 0$

$\text{blkidx.z} = 0$

$\text{blkidx} = (2, 0, 0)$

$\text{threadidx}(t_x, t_y, t_z)$

$\text{block.x} = 8$; $\text{block.y} = 4$

$t_x = \underline{62} \bmod 8 = 6$

$t_y = 62/8 = 7.75 \approx 7 \bmod 4 = 3$

$t_z = 62/(8 \times 4) = 62/32 = 1.938 \approx 1$

$\text{threadidx} = (6, 3, 1)$

16/3/26
→

Atomic & Arithmetic functions

→ In a multithreaded scenario, if multiple threads try to modify a single shared memory variable, the issue of data inconsistency will arise.

→ To overcome this atomic functions need to be used:-

① `atomicAdd()`

② `atomicSub()`

③ `atomicExch()`

eg:-
 $\begin{array}{c} *p, v \\ \downarrow \\ old = *p; \\ \downarrow \\ *p = v; \\ \downarrow \end{array}$

④ `atomicInc()` [Increment]

⑤ `atomicDec()` [Decrement]

→ `cudaEvent_t` $\Rightarrow$ data type to measure time w. variables start & stop.

→ `cudaEventCreate(&start);` $\rightarrow$ `/(&stop);` $\Rightarrow$ Initializing start/stop

→ `cudaEventRecord(start, 0);` $\Rightarrow$ could be before or after malloc: placing
or `cudaEventRecord(stop, 0);` $\Rightarrow$ used to record time

→ `cudaError_t` $\Rightarrow$ for error launch

→ `cudaEventSynchronize(stop);` $\rightarrow$ to get all returned values of threads (as they may have returned abcount but other operations may not have returned)

Explanation

threads 1 2 3 4
↓ ↓ ↓ ↓
may have not completed all work $\Rightarrow$ hence synchronize
count completed!!

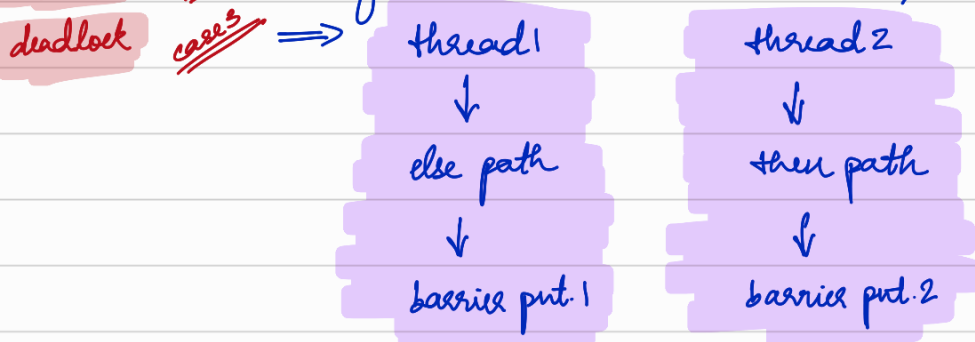
→ `cudaEventElapsedTime` $\Rightarrow$ to return stop-start time to pass on from 'Device to Host' to get the time taken

→ Synchronization & Transparent Scalability

→ `syncthreads()` $\Rightarrow$ a better synchronization barrier function

if-then-else stmt. case 1 $\Rightarrow$ then path and else path : in an if-then-else statements
all threads execute for or all threads execute for

if-else stmt. case 2 $\Rightarrow$ if-else statement : all execute / none



Meaning both will be waiting at diff. barrier synchronization points (deadlock) = they would end up waiting for each other forever (∞)

diff. blks case 4 $\Rightarrow$ these threads (within a blk) should execute in a close time proximity w. each other : threads in diff. blk

23/2/26

→ Matrix Multiplication

Each row of resultant matrix to be computed by one thread

multiply kernel rowwise $\lll 1, \text{ra} \ggg$ (da, db, dc, wa, wb);

global void multiplyKernelRowwise(int* a, int* b, int* c, int wa, wb) {
 int ridA = threadIdx.x;
 int sum;
 for (int cidB; cidB <



→ Each column of resultant matrix to be computed by one thread

* MIDTERM PORTION :- L1 — L21

25/2/26

Q)

Thread index (2, 1)

Blk (0, 1)

global threadId?

∴

blkDim = (8, 4)

gridDim.x = 5

$$tpb = 8 \times 4 = \underline{\underline{32}}$$

$$\text{linear blkid} \Rightarrow \text{blkid} = \text{blkidx.x} + \text{blkidx.y} \times \text{gridDim.x} \\ = 0 + 1 \times 5$$

$$= \underline{\underline{5}}$$

$$\text{linear threadid} \Rightarrow \text{localid} = \text{threadidx.x} + \text{threadidx.y} \times \text{blkDim.x} \\ = 2 + 1 \times 8 \\ = \underline{\underline{10}}$$

$$\therefore \text{globalid} = \text{threadidx.x} + \text{threadidx.y} \times \text{blkDim.x} \\ = 2 + 1 \times 8 \\ =$$

16/3/26

* April 4th $\Rightarrow$ Lab Quiz

Q) Write CUDA Kernel for Tiled 1D Convolution (using shared Memory)

A:

```
#define TILE_WIDTH 16

__global__ void convolution1Dshared (float *N, float *M, float *P, int MaskWidth,
                                     int Width) {

    __shared__ float Ns[TILE_WIDTH + 2];

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int n = MaskWidth / 2;

    if (i < Width)
        Ns[threadIdx.x + n] = N[i];

    if (threadIdx.x < n) {
        if ((i - n) >= 0)
            Ns[threadIdx.x] = N[i - n];

        if ((i + blockDim.x < Width))
            Ns[threadIdx.x + n + blockDim.x] = N[i + blockDim.x];
    }

    __syncthreads();

    float Pvalue = 0;
    if (i < Width) {
        for (int j = 0; j < MaskWidth; j++) {
            Pvalue += Ns[threadIdx.x + j] * M[j];
        }
        P[i] = Pvalue;
    }
}
```

Q) Write CUDA Kernel for Basic 1D Convolution.

Ans: global void convolution1D(float *N, float *M, float *P, int Mask_Width,
int Width){

int i = blockIdx.x * blockDim.x + threadIdx.x;

int start = i - Mask_Width/2;

float Pvalue = 0;

for (int j=0; j<Mask_Width; j++){

if (start+j) >=

Tiled 1D convolution w. Halo Elements

```
global void convolution_1D_basic_kernel (float *N, float *P, int MaskWidth,
                                         int width){
```

```
    int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    shared float Nds[TILE_SIZE + MAX_MASK_WIDTH - 1];
```

```
    int n = MaskWidth/2;
```

```
    int halo_index_left = (blockIdx.x - 1) * blockDim.x + threadIdx.x;
```

```
    if (threadIdx.x) = (blockDim.x - n){
```

```
        Nds[threadIdx.x - (blockDim.x - n)] =
```

```
            (halo_index_left < 0) ? N(halo_index_left);
```

```
    }
```

```
syncthreads();
```

```
float Pvalue = 0;
```

```
for (int j = 0; j < MaskWidth; j++){
```

```
    Pvalue += Nds[threadIdx.x + j] * H[j];
```

```
}
```

```
P[i] = Pvalue;
```

Q) Fill in the table fields to show the state of the shared memory & the output at each step

a)	N = {1, 2, 3, 4, 5, 6}	blk index	Thread ind.	SM by Nds	O/P (P)
Conv. Kernel (K)/mask:		0	0	0, 1, 2	-2
K = [1, 0, -1]		0	1	1, 2, 3	-2
Tile Width : 3		0	2	2, 3, 4	-2
		1	0	3, 4, 5	-2
		1	1	4, 5, 6	-2
		1	2	5, 6, 7	-2

resultant array $\therefore P = [-2, -2, -2, -2, -2, -2]$

b) $N = \{7, 8, 9, 10, 11, 12, 13\}$

(0, 0, 7, 8, 9)

(0, 7, 8, 9, 10)

(7, 8, 9, 10, 11)

(8, 9, 10, 11, 12)

(9, 10, 11, 12, 13)

(10, 11, 12, 13, 0)

(11, 12, 13, 0, 0)

(12, 13, 0, 0, 0)

→ Sequential Sparse-Matrix (Vector Multiplication)

a) Write CUDA Kernel for Tiled Matrix Multiplication.

A: #define TILE_WIDTH 16

```
global void MatrixMultiled (float *A, float *B, float *C, int width) {  
    shared float As[TILE_WIDTH][TILE_WIDTH];  
    shared float Bs[TILE_WIDTH][TILE_WIDTH];
```

int Row

have a
shared mem
not there
(addition)

30/5/26 →

Imp. of Memory Access Efficiency

→ CUDA Device Memory types

→ Constant Memory & Caching

→ Cache coherence



A tiled Matrix-Matrix Multiplication

sample trace

A =

0	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

blk idx thread idx

(0,0) (0,0)

(0,0) (0,1)

(0,0) (1,0)

(0,0) (1,1)

(0,1) (0,0)

(0,1) (0,1)

Matrix d_N:

B =

16	15	14	13
12	11	10	9
8	7	6	5
4	3	2	1

(0,1) (1,0)

(0,1) (1,1)

(1,0) (0,0)

(1,0) (0,1)

(1,0) (1,0)

(1,0) (1,1)

(1,1) (0,0)

(1,1) (0,1)

(1,1) (1,0)

(1,1) (1,1)

Matrix d_P: Partial phase 1

d_P :

40	37	34	31
152	141	130	119
264	245	226	207
376	349	322	295

Title Width: 2x2